Enhancing Vision-Language-Action model with Reward-free Goal-conditioned Reinforcement Learning for Long-Horizon Control

Geunha Kim*, Seongsu Kim*, Junbeom Lee, and Jun Moon†

***Abstract*—Recently, Vision-Language-Action (VLA) models have shown promising generalization across tasks. There have been continued efforts to improve their performance through reinforcement learning (RL), particularly by learning value functions that capture task progress. However, such approaches often struggle with time-consuming reward labeling. To address this limitation, we exploit the unsupervised objective of offline goal-conditioned RL (GCRL) to learn a goal-conditioned value function in an unsupervised manner. Building on this approach, we propose Goal-conditioned Vision-Language-Action (GC-VLA), a framework that harnesses the reward-free objective of offline GCRL to improve VLA models. To this end, we leverage the structural characteristics of VLA models to design a hierarchical architecture for long-horizon control, in which the VLM generates intermediate subgoals and the action model generates action chunks conditioned on the generated subgoals. We evaluate our model on complex long-horizon instruction-following tasks composed of multiple subtasks, in which successful execution requires sequential interaction with the environment. Experimentally, we demonstrate that our model outperforms the baseline models under these challenging conditions by enabling more effective long-horizon control.**

***Index Terms*—Reinforcement learning, Vision-language-action models, Long-horizon control, Manipulation.**

I. INTRODUCTION

Recent advances in large-scale data and expressive models have enabled Vision-Language-Action (VLA) models [4], [5], [8] to address everyday robotic tasks. Despite the high cost of collecting robot data, large-scale datasets gathered across diverse environments and platforms [1], [3] have significantly improved the generalization ability of VLA models. To further enhance performance, recent works have explored reinforcement learning (RL)-based fine-tuning on top of behavior-cloned VLA policies [7], [10], [11]. However, approaches that rely on online interaction remain costly and inefficient, whereas methods based on explicit reward labels require time-consuming annotation and face limitations when scaling to multiple tasks.

These limitations point to the need for learning approaches that do not rely on explicit reward supervision. In computer vision and natural language processing, unsupervised learning paradigms such as next-token prediction [13], masked auto-encoding [14], and contrastive learning [15] have demonstrated strong general performance across diverse tasks. Offline GCRL [16], [17] follows the same principle: rather than relying on manually specified rewards, it treats future states encountered in the dataset as self-supervised goal signals, analogous to how masked auto-encoding reconstructs unlabeled pixels to learn visual structure — both paradigms extract supervisory signal directly from the data itself. In reinforcement learning, offline GCRL aligns well with this direction, as it enables learning from diverse offline dataset without requiring reward labeling. Offline GCRL operates in an unsupervised manner by treating states observed in the dataset as goals and learning policies conditioned on them, thereby avoiding domain-specific assumptions while learning to reach diverse states contained in the dataset.

Inspired by these benefits, in this paper, we propose incorporating offline GCRL into VLA models. Our approach begins with pretraining a goal-value function on diverse offline datasets using Implicit Q-Learning [18] to learn a unified representation that measures temporal distance [19], [20] between states and goals. Such embedding distances reflect the number of steps separating two states, providing a task-agnostic measure of progress that is directly usable for subgoal prediction — preserving spatial and geometric precision that image-space or language-based alternatives struggle to maintain.

Building on this representation, we train the VLM component of the VLA to predict subgoal representations conditioned on the current observation, instruction and terminal goal. Each subgoal corresponds to a future state embedded by the goal-value function. For adaptation to a target environment, we fine-tune both the goal-value function and the VLM using the target dataset. We train a low-level policy that generates action chunks [6] conditioned on the current observation and the predicted subgoal representation. We use the advantage conditioning [21], [22] based on the goal-value estimates to guide action selection.

We evaluate our models in both simulation and real-world environments. In simulation, we validate the effectiveness of the proposed hierarchical architecture and advantage conditioning across two different environments. In real-world, our model achieves higher success rates and requires fewer execution steps than the baseline, indicating more accurate subtask inference during long-horizon instruction following. Moreover, in tasks with multiple possible choices, our model demonstrates more general decision behavior by selecting among alternatives in a flexible manner. These results demonstrate the effectiveness of our model in long-horizon control. The main contributions of this work are as follows:

- We propose **GC-VLA**, a framework that improves VLA

*Equal contribution.
†Corresponding author.

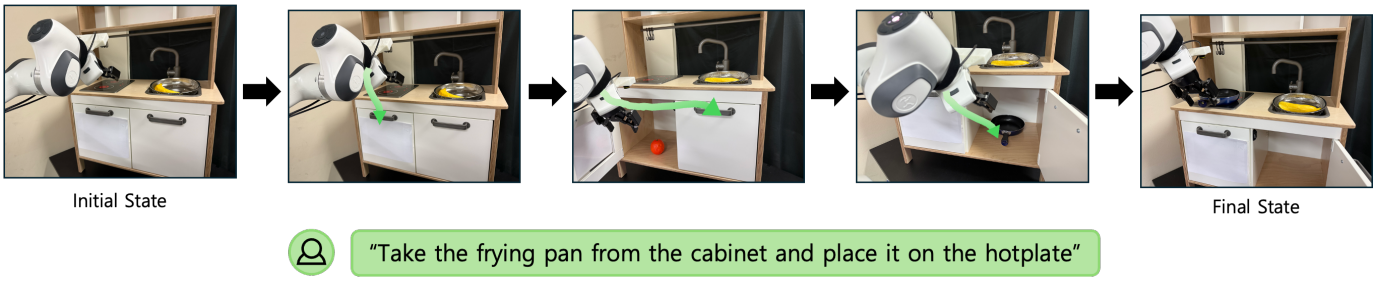


Fig. 1. **Task example.** A robot must complete multiple subtasks to successfully execute the instruction, interacting with the environment to reveal the target, and then grasp it. Since the instruction does not specify the exact location of the target, the task requires long-horizon control through sequential interaction with the environment.

models by learning goal-conditioned values that represent task progress, using reward-free objective of offline GCRL without explicit reward labeling.
- For long-horizon control, we leverage the hierarchical structure of VLA models, where the VLM generates subgoal representations and the action model produces action chunks conditioned on both the learned value and generated subgoals.
- We demonstrate our model outperforming the baseline VLA model on complex long-horizon instruction-following tasks, which require interaction with environment as illustrated in Figure 1.

II. Related works

A. Foundation Models in Robotics

Recent advances in foundation models have improved generality in robotics. In particular, vision-language-action models (VLAs) [6], [8], [9], [12] leverage web-scale data to combine multimodal information into real-world actions. Transformer-based approaches [9] model action sequence by treating trajectories as token sequences. LLM/VLM-based methods [23] treat VLA tasks as sequence-to-sequence problems for action generation. Diffusion-based methods [24] generate actions through iterative denoising diffusion models. On the other hand, rather than the model architecture, many recent works have studied how to improve a base VLA model through RL. Several works directly apply the offline RL or online RL thereof to VLA fine-tuning, yielding approaches that are difficult to extend to real-world RL in an efficient and general fashion. In contrast, we adopt a goal-conditioned RL that enables using an unlabeled dataset efficiently, thereby avoiding explicit reward annotation and making the RL setting more practical in real-world rollouts.

Existing VLAs largely follow two paradigms: 1) end-to-end policies that directly map images and language to low-level actions, and 2) hierarchical formulations in which a high-level policy exploits images and language information to produce an intermediate representation that is decoded into actions by a low-level policy. Our work builds on the latter paradigm by using the high-level policy to generate subgoal representations, which are subsequently used to condition the low-level policy for execution.

B. Offline Goal-conditioned RL

Offline goal-conditioned RL (GCRL) learns goal-conditioned policies from reward-free data by treating future states along trajectories as self-supervised goals via hindsight relabeling, removing the need for hand-specified rewards and enabling reuse of large, unlabeled datasets. Prior works realize this through contrastive [25], [26], quasimetric [27], and metric based [28], [29] value functions that capture the temporal distance between states and goals. This progress signal has been extended to long-horizon tasks via high-level subgoal search [37], to constrained settings via recovery policies [38], and to denser supervision via graph-based reward shaping [39]. However, these methods operate mainly in low-dimensional state or single-domain settings and do not address integration with large pretrained policies.

A complementary line studies hierarchical VLA (Hi-VLA) systems, where a high-level VLM planner decomposes a task into subgoals for a low-level controller; a recent study [40] shows that the subgoal interface and the planner's observation/memory representation are decisive for performance. Yet most Hi-VLA planners communicate through discrete *language* subgoals, which lose the geometric precision needed for fine-grained search under occlusion. We connect these two lines: GC-VLA keeps the reward-free objective of offline GCRL but expresses subgoals as *continuous Hilbert-space representations*, has the VLM predict them directly rather than search over them [37], and uses the resulting goal-value estimates as an advantage signal for the low-level policy.

III. Methodology

Problem setting. We consider a Markov decision process $\mathcal{M} = (\mathcal{O}, \mathcal{A}, \mu, p, r)$, where $o_t \in \mathcal{O}$ is the observation space, $a_t \in \mathcal{A}$ is the action space, $\mu \in \mathcal{P}(\mathcal{O})$ is an initial distribution, $p \in \mathcal{O} \times \mathcal{A} \to \mathcal{P}(\mathcal{O})$ is a transition distribution, and $r(o, g)$ is a goal-conditioned reward function that returns 0 if $o = g$, and -1 otherwise. We denote the reward by r_t for simplicity. We assume two datasets $\mathcal{D}$ and $\mathcal{D}_o$, where $\mathcal{D}$ consists of robot action trajectories $\tau = (o_0, a_0, o_1, a_1, \cdots, o_T)$, while $\mathcal{D}_o$ consists solely of observation trajectories $\tau_o = (o_1, o_2, \cdots, o_T)$. We consider the goal space $\mathcal{G}$ to coincide with the observation space, meaning that every observation in the entire dataset $\mathcal{D} \cup \mathcal{D}_o$ can be treated as a goal. Our

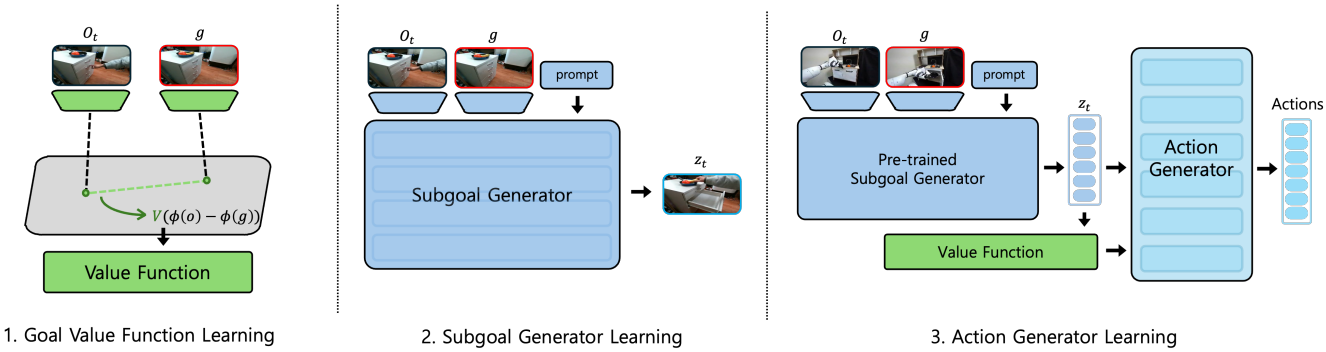


Fig. 2. **Overview of GC-VLA.** 1) Goal Value Function Learning: We first learn a goal-value function which measures the embedding distance between current and goal observations in the Hilbert space. 2) Subgoal Generator Learning: The goal-conditioned VLM is trained to predict subgoals, which are used as conditioning inputs to the action generator. 3) Action Generator Learning: Conditioned on the predicted subgoals, a flow-matching-based policy is trained to predict action chunks for execution. All three stages are pretrained on diverse datasets and then fine-tuned on the target robot data.

goal is to train optimal goal-conditioned policy $\pi(a|o, g)$ by using $\mathcal{D} \cup \mathcal{D}_o$ to maximize the cumulative goal-conditioned reward $\mathcal{J}(\pi) = \mathbb{E}_{g\sim p(g),\tau\sim p^\pi(\tau)}\left[\sum_{t=0}^{T} \gamma^t r(o_t, g)\right]$, where $p^\pi(\tau) = \mu(o_0)\Pi_{t=0}^{T-1}\pi(a_t|o_t, g)p(o_{t+1}|o_t, a_t)$, $p(g)$ is a goal distribution and γ is a discount factor.

Our method is organized into three components. Learning Hilbert Representations (III-A) learns a temporal-distance metric between the current and the goal observation in a Hilbert space $\mathcal{Z}$ from reward-free data, facilitating effective VLA learning without reward annotations. Learning Goal-conditioned High-level Policy (III-B) then generates subgoal representations from the current observation to support efficient control. Finally, Learning Goal-conditioned Low-level Policy (III-C) executes action chunks conditioned on the observation and the generated subgoal to reach the intermediate objective. Figure 2 provides the pipeline overview.

A. Learning Hilbert Representation

We start by training a representation function $\phi : \mathcal{O} \to \mathcal{Z}$ that embeds observations into a latent space, where embedding distances correspond to temporal distances. Our main desideratum is to train a single representation function that consistently represents temporal distance across heterogeneous data, including human demonstrations and robot data from diverse environments. This design is motivated by the need for a high-level policy that predicts subgoal representations consistently across diverse data. A single shared representation function allows for consistent subgoal prediction without introducing additional encoders that would increase computational complexity. To achieve this objective, we adopt Hilbert space $\mathcal{Z}$ representation learning formulation proposed in Foundation Policies with Hilbert Representations (HILP) [29].

Following the HILP, we model the goal-value function as the distance between embedded states in a Hilbert space,

$$d^*(o, g) = \|\phi(o) - \phi(g)\| \tag{1}$$

where ϕ is a Hilbert representation and $d^*(o, g)$ measures the temporal proximity between states and goals. We train ϕ such that the optimal goal-conditioned value function satisfies $V^*(s, g) = -d^*(o, g)$. To train the optimal value function, we adopt an action-free IQL-style objective, where temporal-difference targets are defined using the value function:

$$\max_{\omega} \mathbb{E}_{o_t,o_{t+1},g}\left[\ell_\tau^2(r(o_t, g) + \gamma\hat{V}_{\hat{\omega}}(o_{t+1}, g) - V_\omega(o_t, g))\right] \tag{2}$$

where $r(o, g) = -\mathbf{1}(o \neq g)$ denotes a reward function, $\ell_\tau^2(u) = |\tau - \mathbf{1}(u < 0)|u^2$ denotes the expectile loss, γ denotes a discount factor, $\hat{V}$ denotes the target network. In practice, we parameterize the value function as $V(o, g) = -||\phi(o) - \phi(g)||$ and substitute this form into (2) to train ϕ.

B. Learning Goal-conditioned High-level policy

Our instruction set is intentionally underspecified with respect to the target's precise location under occlusion. With these instructions, the policy must infer task-relevant object location information and execute the instruction following task. In this setting, approaches that directly map observations and language to immediate actions at each time step may exhibit a diffuse action distribution before discovering the target, since multiple exploratory behaviors can appear. Such dispersion can undermine long-horizon performance.

To address this challenge and enable effective planning in complex long-horizon tasks, we introduce a subgoal generator within a goal-conditioned VLM. Specifically, we adopt a goal-conditioned formulation that jointly conditions on the current observation o_t, goal g, and instruction l, and maps these inputs to a subgoal representation z_t aligned with progress toward goal completion. To define the subgoal target during training, we introduce a subgoal horizon k, which determines how far ahead in the trajectory the subgoal is set. Concretely, given the current timestep t and the final goal timestep t_g, the subgoal target is defined as the observation at timestep $\min(t + k, t_g)$. This formulation ensures that the predicted subgoal always lies between the current state and the final goal along the trajectory, providing a reachable intermediate objective that bridges the gap between immediate observations and long-horizon task completion. The objective is as follows:

$$\max_{\psi} \mathbb{E}_{(o_t,g,l)\sim\mathcal{D}}[\log \pi_\psi^h(z_t|o_t, g, l)] \tag{3}$$

This is intended to extract instruction-following intermediate objectives (i.e. subgoals) that facilitate exploration and

Algorithm 1 Goal-conditioned VLA (GC-VLA)

Require: Offline heterogeneous dataset $\mathcal{D}_h = \{(o_t, o_g, l)\}$
Require: Offline target robot dataset $\mathcal{D}_r = \{(o_t, o_g, a_t)\}$
Train V_{pre} on D_h using Eq. (2)
Train π^h_{pre} on D_h using Eq. (3)
while not converged **do**
    Sample $\{(o_t, o_g)\}_{i=1}^B \sim \mathcal{D}_r$
    Train V from V_{pre} by using Eq. (2)
end while
while not converged **do**
    Sample $\{(o_t, o_g, l)\}_{i=1}^B \sim \mathcal{D}_r$
    Train π^h from π^h_{pre} by using Eq. (3)
end while
while not converged **do**
    Sample $\{(o_t, z_t, a_t)\}_{i=1}^B \sim \mathcal{D}_r$
    Train π^l by using Eq. (5)
end while

structured decision making in visually ambiguous scenes. We represent subgoals as embeddings in the Hilbert space defined by the learned value function, where distances directly reflect temporal proximity between states. Both observations and subgoals are projected into a shared space during training and inference, enabling subgoal prediction to operate on a consistent, aligned representation.

To enable effective subgoal inference in our setting, we reformulate the standard VLM architecture into a goal-conditioned setting. We use PaliGemma [2] as our model backbone. It combines the Transformer-based language backbone Gemma-2b [30] with a ViT encoder SigLIP [31]. We condition the backbone on the current observation, the terminal goal image, and the instruction, and train it to regress the subgoal representation z_t in the Hilbert space.

C. Learning Goal-conditioned Low-level Policy

Given the predicted subgoal z_t from the high-level policy, the low-level policy generates action chunks conditioned on the current observation and z_t. Recent VLA models adopt flow-matching policies, which are difficult to optimize directly with RL due to their iterative generative structure [8]. While one-step flow models [32] optimize a value function $Q(s, a)$ with a BC regularizer, adding such a policy on top of an already high-capacity VLA substantially increases computational and memory cost. Instead, we adopt a variant of advantage conditioning: the policy is trained by supervised learning, augmented with an extra input that indicates the optimality of each action as measured by its advantage.

We extract the policy via regularized RL [33], maximizing the expected return while staying close to a reference policy π_{ref}. Following the regularized policy extraction in $\pi^*_{0.6}$ [21], the KL-regularized solution takes the form

$$\hat{\pi}(a|o, z_t) \propto \pi_{\text{ref}}(a|o, z_t)\, \exp\big(A^{\pi_{\text{ref}}}(o, a, z_t)/\beta\big), \qquad (4)$$

where the advantage $A^{\pi_{\text{ref}}}(o_t, a_t, z_t) = V(o_{t+1}, z_t) - V(o_t, z_t)$ is derived from our goal-conditioned value function. Applying Bayes' rule under a uniform prior on an optimality indicator I and taking $\beta = 1$, this reduces to $\hat{\pi}(a|o, z_t) = \pi_{\text{ref}}(a|I, o, z_t)$. Assuming $I_t = \mathbf{1}(A^{\pi_{\text{ref}}}(o_t, a_t, z_t) > \epsilon)$ with an improvement threshold ϵ, we obtain the policy objective

$$\max_\theta \mathbb{E}_{\mathcal{D}} \left[\log \pi_\theta(a_t|o_t, z_t) + \alpha \log \pi_\theta(a_t|I_t, o_t, z_t)\right], \qquad (5)$$

where α is a trade-off parameter. The low-level policy is thus trained both with and without the advantage indicator I_t, combining behavior cloning with advantage-guided refinement. The overall procedure is summarized in Algorithm 1.

IV. Experiments

To evaluate GC-VLA, our experiments are organized into two parts. First, we conduct simulation experiments to validate the effectiveness of the hierarchical architecture, examining whether the generated subgoals reflect meaningful progress toward the final goal and whether advantage conditioning contributes to structured behavior. Second, we evaluate GC-VLA in real-world long-horizon tasks that require sequential interaction with the environment, demonstrating that our model achieves superior performance over the baselines under these challenging conditions.

A. Task Setups

As illustrated in Figure 3, we evaluate our method in four different tasks, spanning both real-world and simulated setups. The tasks are grouped into real tasks: **Kitchen-F**, which requires finding a frying pan inside the cabinet and placing it on the hotplate, and **Kitchen-B**, which requires placing a banana onto a plate occluded within the cabinet.

In the real-world setting, we use three metrics to evaluate the search behavior. First, **Search** measures whether the policy successfully finds the target object. Second, **Complete** reports the rate at which the policy successfully places the target. Finally, **Steps** denotes the number of inference steps. Search is performed over two cabinets.

To validate the effectiveness of our method, we additionally include simulated tasks: **Simulation-S**, which focuses on searching the target object under occlusion in simulation, and **Simulation-P**, which requires placing the object at the correct location on an occluded plate inside the cabinet. In the simulated tasks, search is performed over four cabinets, and each **Task** in the table indicates the location of the target.

B. Datasets and Baselines

Datasets. We construct a heterogeneous training corpus that combines a large-scale robot dataset, a human-collected dataset, and our target-robot dataset (Figure 4). The goal-value function and the high-level policy are pretrained on the human dataset, augmented with DROID [1], from which we filter trajectories relevant to our setting (e.g., cabinet-opening). Pretraining on this combined corpus allows the high-level policy to generate subgoals in a task-agnostic manner before adaptation. We then fine-tune the pretrained goal-value function and high-level policy on our target-robot data, whereas the low-level policy is trained directly on the target-robot data without pretraining. In total, we use 560k steps from

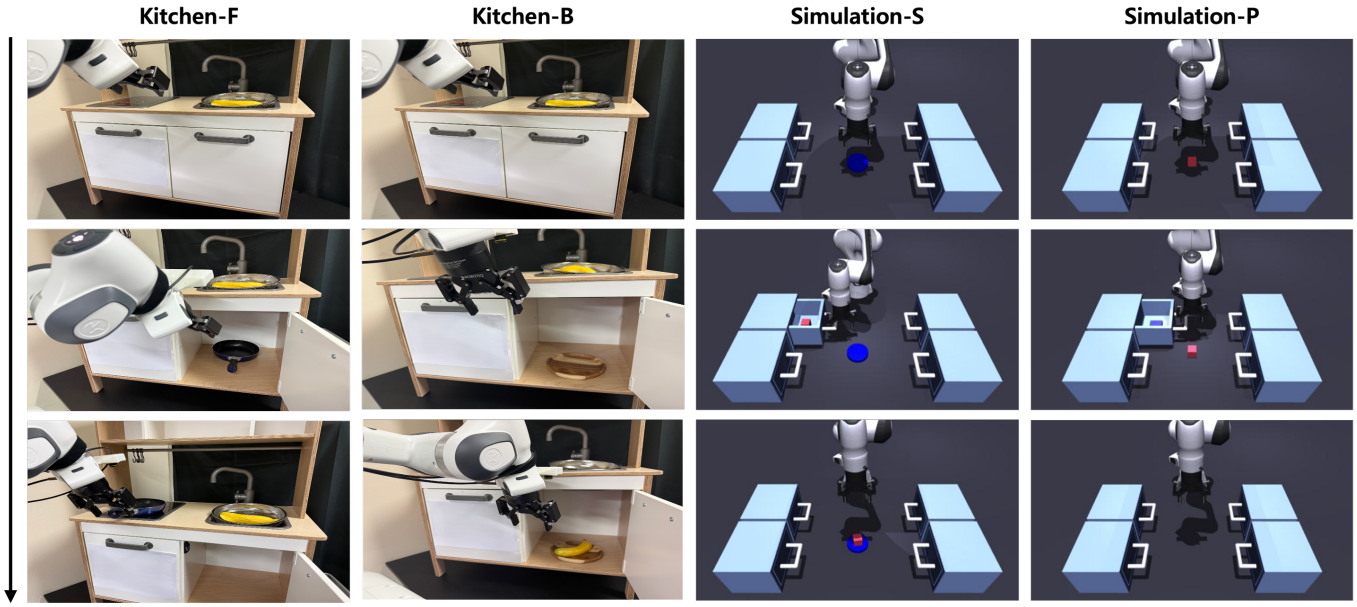


Fig. 3. **Environments.** We set up 4 different environments to evaluate the tasks. The left two columns are real and right two columns are simulation.

TABLE I
SUCCESS RATE ON SIMULATION TASKS ACROSS ARCHITECTURAL VARIANTS

Environment	Task	Flat BC	HBC	HBC w/ AC-Low (ours)
Simulation-S	Task 1	30 ± 12	26 ± 5	**35** ± 4
	Task 2	26 ± 10	**32** ± 4	29 ± 10
	Task 3	15 ± 4	7 ± 4	**62** ± 2
	Task 4	10 ± 9	35 ± 7	**40** ± 12
	Overall	20 ± 9	25 ± 13	**42** ± 14
Simulation-P	Task 1	41 ± 12	30 ± 10	**48** ± 13
	Task 2	9 ± 6	13 ± 10	**18** ± 8
	Task 3	12 ± 4	22 ± 13	**38** ± 3
	Task 4	27 ± 9	30 ± 5	**33** ± 6
	Overall	22 ± 15	24 ± 8	**34** ± 12

TABLE II
SUCCESS RATE ON REAL TASKS

Model	Kitchen-F			Kitchen-B		
	Search ↑	Complete ↑	Steps ↓	Search ↑	Complete ↑	Steps ↓
BC	45.0 ± 19.1	40.0 ± 24.5	747	37.5 ± 19.3	37.5 ± 19.3	721
HGCBC	50.0 ± 5.8	42.5 ± 17.1	755.6	45.0 ± 19.1	40.0 ± 16.3	702
π_0	32.5 ± 9.6	27.5 ± 15.0	756.3	40.0 ± 29.4	22.5 ± 20.6	638
GC-VLA (ours)	**75.0 ± 5.8**	**52.5 ± 9.6**	**722.6**	**82.5 ± 12.6**	**42.5 ± 22.2**	**622**

the large-scale dataset, 70k steps from the human dataset, and 18k steps from our target-robot dataset.

Baselines. Because the simulated and real-world settings differ substantially in action- and observation-space complexity, we adopt setting-specific baselines while keeping the underlying hierarchical formulation fixed. In simulation, where the action and observation spaces are smaller, we reduce both the dataset size and the model capacity (architectures and hyperparameters in Appendix V-A) and compare three variants: (1) flat behavior cloning (**Flat BC**); (2) hierarchical goal-conditioned BC (**HBC**); and (3) the same hierarchy with advantage conditioning applied to the low-level policy (**HBC w/ AC-Low**), which is the simulation instantiation of our GC-VLA. In the real world, we compare against (1) **BC**, which performs flat imitation learning on the target-robot data; (2) **HGCBC**, a hierarchical goal-conditioned BC in which the high-level policy predicts raw future-index subgoals and the low-level policy is behavior-cloned conditioned on them—unlike GC-VLA, HGCBC learns no structured temporal-distance representation and thus provides no explicit measure of task progress; and (3) π_0 [8], a generalist manipulation foundation policy initialized from its pretrained weights and fine-tuned on our target-robot data. All baselines are trained on the same target-robot data, serving as strong references for the tasks we construct.

C. Architecture Validation in Simulation

In simulation, we categorize tasks into four variants (1-4) based on the target object placement in the cabinet. At each environment, there are four cabinets, and the target object may be located in any one of them depending on the task variant. One of the main points of the simulation tasks is sequential control. In practice, both flat BC and hierarchical BC often fail because the policy repeatedly opens the same cabinet rather than progressing to other cabinets. This behavior prevents the policy from completing the required sequential search process. As shown in Table I, both BC and HBC achieve relatively low success rates across the task variants. In contrast, our method with advantage conditioning applied to the low-level policy performs more effective search behaviors, allowing the policy to consistently follow the sub-goals and successfully find the target object within the given timestep. Reported values are the mean success rate ± standard deviation over 4 seeds.

D. Real-world Results

In the real-world tasks, we compare our approach against three baselines as mentioned above: BC, HGCBC and π_0. Experiments are conducted in the Kitchen-F and Kitchen-B environments. Before evaluating our method, we observe that baselines exhibit limited search competence under occlusion. In particular, mapping the current observation directly to actions provides no mechanism to commit to and sequence intermediate subgoals: after completing one subtask, baselines often fail to transition to the next step. During rollouts, this limitation manifests as low-value exploration, such as repeatedly re-opening cabinets that have already been explored. Consequently, baselines exhibit low success rates in search tasks.

By contrast, as shown in Table II, GC-VLA demonstrates substantially higher search success and completion rates relative to the baselines. We evaluate from an initial observation in which the target is not visible, corresponding to the object being inside the cabinet. In Kitchen-F, we report success under two goal specifications: one where the target (Frying pan) is occluded by the robot arm, and another remains visible. Further, in evaluation, we designate the final goal as an intermediate subgoal within the both same task. We observe similar strong performance, supporting that the instruction-to-subgoal mapping is learned effectively during training. The largest gains appear in search success, where our method must reason about where the occluded target is; completion further depends on the subsequent grasp-and-place, a manipulation skill shared across all methods. Overall, these results show that our method achieves reliable search and execution without explicit reward labeling.

GC-VLA also completes long-horizon rollouts in fewer steps than the baselines, which frequently incur additional steps due to weak linkage between subtasks. The number of inference steps remains comparable across left and right cabinet placements, indicating that efficiency is consistent regardless of target side. Furthermore, the policy does not exhibit a fixed preference for which cabinet to open first, suggesting that the learned search strategy does not rely on a fixed opening order, though we did not evaluate held-out spatial configurations. GC-VLA also shows strong recovery behavior: when the target is missed, the policy re-attempts the grasp.

For evaluation, each task is tested under two target placements, with 20 trials for each of 4 random seeds, and we report the mean success rate; the reported $\pm$ values denote the standard deviation across the 4 seeds. Steps are reported only as a coarse efficiency indicator and are not directly comparable across rows.

V. CONCLUSIONS

In this paper, we introduce Goal-conditioned Vision-Language-Action models by incorporating offline goal-conditioned RL into the VLA framework. By leveraging its unsupervised learning approach, our method allows both pretraining and fine-tuning to be conducted without explicit reward labeling. We first pretrain the models on heterogeneous datasets and then fine-tune them on target environment data under the same training procedure. Then, we evaluate GC-VLA on long-horizon tasks that require solving multiple subtasks, where prior VLA models typically struggle, and demonstrate superior performance. Although our experiments cover heterogeneous datasets, they do not yet extend to internet scale, leaving open whether our approach generalizes to a wider range of tasks. In addition, while our simulation study validates the hierarchical structure and advantage conditioning, it does not isolate the contribution of the Hilbert representation itself; a controlled ablation that varies only the subgoal representation is beyond the scope of this work and remains an important direction for future work. Scaling the training data and conducting such component-level analyses are left for future study.

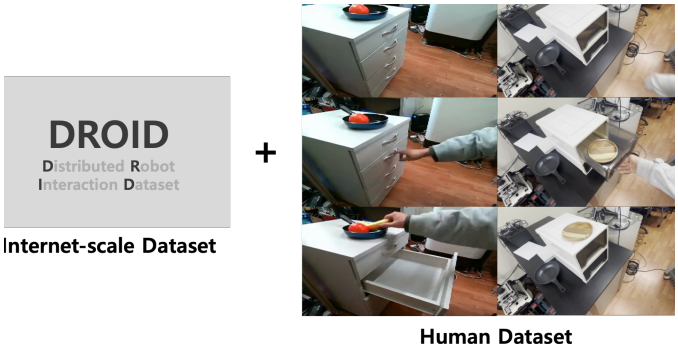


Fig. 4. **Datasets.** We build a heterogeneous dataset by combining DROID with our human dataset.

APPENDIX

A. Network Details

We conducted experiments in both simulation and real-world environments, employing different network architectures for each setting. As the simulation tasks are simpler than the real-world tasks, we designed distinct network architectures for each environment. In the simulation setting, all image observations were encoded using a smaller variant of the IMPALA [34] encoder, followed by a multi-layer perceptron (MLP). The policy network for action generation consisted of a two-layer residual MLP architecture.

B. Implementation Details

To enable a generalizable subgoal representation in our VLA framework, we first pretrain the goal-conditioned value function and the VLM. The goal-conditioned value function is trained using the (2), where goals are sampled from a mixture of random dataset observations and future observations along trajectories. We relabel goals and targets using the same relabeling strategy as HILP. Random goals are drawn uniformly from the dataset, while future goals are sampled from a geometric distribution with parameter $(1-\gamma)$.

The sampling probabilities are 0.375 for random goals and 0.635 for future goals. After pretraining the Hilbert representation ϕ, we leverage it to pretrain the high-level VLM using (3). For the high-level policy, a goal g is sampled uniformly from

TABLE III
HYPERPARAMETERS FOR THE SIMULATION SETTING

Hyperparameter	Value
Optimizer	AdamW [35]
Learning rate	3e-4
Weight decay	1e-6
Gradient step	5×10^5
Minibatch size	256
Value MLP dimensions	(512, 512, 512)
High-level Policy MLP dimensions	(512, 512, 512)
Low-level Policy MLP dimensions	(512, 512, 512)
Activation function	GELU [36]
Target network smoothing coefficient	0.005
Discount factor (γ)	0.99
Flow steps	30
Flow time sampling distribution	Unif($[0, 1]$)
Hilbert representation expectile	0.95
subgoal step	25
Latent dims	32
Action chunk of length	16
Inference step	4

future observations t_g in the same trajectory (with timestep $t_g > t$), and the target is defined as the observation at timestep $\min(t + k, t_g)$, where k is a subgoal step parameter. For the low-level policy, the subgoal is defined as the observation at timestep $\min(t + k, T)$ within the same trajectory, where T denotes the terminal timestep. Our low-level policy generates action chunks of length H, where H is smaller than the subgoal horizon k. We then fine-tune the models using the target environment dataset. We then fine-tune these models on the target environment data under the same training procedure as in pretraining.

C. Hardware setup

As shown in Figure 5, we conduct real-world experiments using a Franka Research 3 with a Robotiq 2F-85 gripper, equipped with two cameras: a ZED 2i mounted on the side and a wrist-mounted camera. The high-level subgoal generator is conditioned only on the side-camera observation, while the low-level action policy additionally uses the wrist-camera view for fine-grained manipulation.



Fig. 5. **The robot setup used in our experiments.** Our manipulation platform in the real-world is equipped with a side and wrist camera.

REFERENCES

[1] Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriany, Mohan Kumar Srirama, Lawrence Yunliang Chen, Kirsty Ellis, et al. Droid: A large-scale in-the-wild robot manipulation dataset. arXiv preprint arXiv:2403.12945, 2024.

[2] Lucas Beyer, Andreas Steiner, André Susano Pinto, Alexander Kolesnikov, Xiao Wang, Daniel Salz, Maxim Neumann, Ibrahim Alabdulmohsin, Michael Tschannen, Emanuele Bugliarello, et al. Paligemma: A versatile 3b vlm for transfer. arXiv preprint arXiv:2407.07726, 2024.

[3] Abby O'Neill, Abdul Rehman, Abhinav Gupta, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, et al. Open x-embodiment: Robotic learning datasets and rt-x models. arXiv preprint arXiv:2310.08864, 2023.

[4] Scott Reed, Konrad Zolna, Emilio Parisotto, Sergio Gomez Colmenarejo, Alexander Novikov, Gabriel Barth-Maron, Mai Gimenez, Yury Sulsky, Jackie Kay, Jost Tobias Springenberg, et al. A generalist agent. arXiv preprint arXiv:2205.06175, 2022.

[5] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, et al. Octo: An open-source generalist robot policy. arXiv preprint arXiv:2405.12213, 2024.

[6] Tony Z Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. arXiv preprint arXiv:2304.13705, 2023.

[7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. CoRR, abs/1706.03762, 2017. URL http://arxiv. org/abs/1706.03762.

[8] K. Black, N. Brown, D. Driess, et al., "π_0: A vision-language-action flow model for general robot control," https://arxiv.org/abs/2410.24164, 2024.

[9] Suneel Belkhale, Tianli Ding, Ted Xiao, Pierre Sermanet, Quon Vuong, Jonathan Tompson, Yevgen Chebotar, Debidatta Dwibedi, and Dorsa Sadigh. RT-H: action hierarchies using language. CoRR, abs/2403.01823, 2024.

[10] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. Advances in Neural Information Processing Systems, 33:6840–6851, 2020. 2020.

[11] Jijia Liu, Feng Gao, Bingwen Wei, Xinlei Chen, Qingmin Liao, Yi Wu, Chao Yu, and Yu Wang. What can rl bring to vla generalization? an empirical study. arXiv preprint, arXiv:2505.19789, 2025.

[12] Haozhan Li, Yuxin Zuo, Jiale Yu, Yuhao Zhang, Zhaohui Yang, Kaiyan Zhang, Xuekai Zhu, Yuchen Zhang, Tianxing Chen, Ganqu Cui, Dehui Wang, Dingxiang Luo, Yuchen Fan, Youbang Sun, Jia Zeng, Jiangmiao Pang, Shanghang Zhang, Yu Wang, Yao Mu, Bowen Zhou, and Ning Ding. Simplevla-rl: Scaling vla training via reinforcement learning. arXiv preprint, arXiv:2509.09674, 2025.

[13] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), 2019.

[14] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Doll'ar, and Ross B. Girshick. Masked autoencoders are scalable vision learners. In IEEE/CVF Computer Vision and Pattern Recognition Conference (CVPR), 2022.

[15] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey E. Hinton. A simple framework for contrastive learning of visual representations. In International Conference on Machine Learning (ICML), 2020.

[16] Seohong Park, Kevin Frans, Deepinder Mann, Benjamin Eysenbach, Aviral Kumar, and Sergey Levine. Horizon reduction makes rl scalable. arXiv preprint arXiv:2506.04168, 2025.

[17] Grace Liu, Michael Tang, and Benjamin Eysenbach. A single goal is all you need: Skills and exploration emerge from contrastive rl without

rewards, demonstrations, or subgoals. In International Conference on Learning Representations (ICLR), 2025.

[18] I. Kostrikov, A. Nair, and S. Levine. Offline reinforcement learning with implicit q-learning. In International Conference on Learning Representations (ICLR), 2022.

[19] Vivek Myers, Chongyi Zheng, Anca Dragan, Sergey Levine, and Benjamin Eysenbach. Learning Temporal Distances: Contrastive Successor Features Can Provide a Metric Structure for Decision-Making. International Conference on Machine Learning, 2024.

[20] Seohong Park, Deepinder Mann, Sergey Levine. Dual Goal Representations. In International Conference on Learning Representations (ICLR), 2026.

[21] Physical Intelligence, $\pi^*_{0.6}$: a vla that learns from experience. arXiv preprint arXiv:2511.14759, 2025.

[22] Kevin Frans, Seohong Park, Pieter Abbeel, and Sergey Levine. Diffusion guidance is a controllable policy improvement operator. arXiv preprint arXiv:2505.23458, 2025.

[23] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, et al., "Do as i can, not as i say: Grounding language in robotic affordances," arXiv preprint arXiv:2204.01691, 2022.

[24] Junjie Wen, Minjie Zhu, Yichen Zhu, Zhibin Tang, Jinming Li, Zhongyi Zhou, Chengmeng Li, Xiaoyu Liu, Yaxin Peng, Chaomin Shen, et al. Diffusion-vla: Generalizable and interpretable robot foundation model via self-generated reasoning. arXiv preprint arXiv:2412.03293, 2024

[25] Chongyi Zheng, Ruslan Salakhutdinov, and Benjamin Eysenbach. Contrastive difference predictive coding. In International Conference on Learning Representations (ICLR), 2024.

[26] Benjamin Eysenbach, Tianjun Zhang, Ruslan Salakhutdinov, and Sergey Levine. Contrastive learning as goal-conditioned reinforcement learning. In Neural Information Processing Systems (NeurIPS), 2022.

[27] Tongzhou Wang, Antonio Torralba, Phillip Isola, and Amy Zhang. Optimal goal-reaching reinforcement learning via quasimetric learning. In International Conference on Machine Learning (ICML), 2023.

[28] Seohong Park, Dibya Ghosh, Benjamin Eysenbach, and Sergey Levine. Hiql: Offline goal-conditioned rl with latent states as actions. In Neural Information Processing Systems (NeurIPS), 2023.

[29] Seohong Park, Tobias Kreiman, and Sergey Levine. Foundation policies with hilbert representations. In International Conference on Machine Learning (ICML), 2024b.

[30] Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, et al. Gemma: Open models based on gemini research and technology. arXiv preprint arXiv:2403.08295, 2024.

[31] Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid loss for language image pre-training. In Proceedings of the IEEE/CVF International Conference on Computer Vision, pages 11975–11986, 2023.

[32] David McAllister, Songwei Ge, Brent Yi, Chung Min Kim, Ethan Weber, Hongsuk Choi, Haiwen Feng, and Angjoo Kanazawa. Flow matching policy gradients, 2025.

[33] Abbas Abdolmaleki, Jost Tobias Springenberg, Yuval Tassa, Remi Munos, Nicolas Heess, and Martin Riedmiller. Maximum a posterior policy optimization. In International Conference on Learning Representations, 2018.

[34] Lasse Espeholt, Hubert Soyer, Remi Munos, Karen Simonyan, Volodymir Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, et al. Impala: Scalable distributed deep-rl with importance weighted actor-learner architectures. arXiv preprint arXiv:1802.01561, 2018.

[35] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. arXiv preprint arXiv:1711.05101, 2017.

[36] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). ArXiv, abs/1606.08415, 2016.

[37] Jinning Li, Chen Tang, Masayoshi Tomizuka, and Wei Zhan. Hierarchical planning through goal-conditioned offline reinforcement learning. *IEEE Robotics and Automation Letters (RA-L)*, 7(4):10216–10223, 2022.

[38] Chenyang Cao, Zichen Yan, Renhao Lu, Junbo Tan, and Xueqian Wang. Offline goal-conditioned reinforcement learning for safety-critical tasks with recovery policy. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.

[39] Lina Mezghani, Sainbayar Sukhbaatar, Piotr Bojanowski, Alessandro Lazaric, and Karteek Alahari. Learning goal-conditioned policies offline with self-supervised reward shaping. In *Conference on Robot Learning (CoRL)*, 2023.

[40] Jiaheng Hu, et al. What matters in orchestrating robot policies: A systematic study of hierarchical VLA agents. arXiv preprint arXiv:2606.10267, 2026.